Module 47

Transactions/2: Serializability

Partha Pratim Das

Department of Computer Science and Engineering
Indian Institute of Technology, Kharagpur
ppd@cse.iitkgp.ac.in

March 18, 2026

Module Recap

- A task in a database is done as a transaction that passes through several states

Module Recap

- A task in a database is done as a transaction that passes through several states
- Transactions are executed in concurrent fashion for better throughput

Module Recap

- A task in a database is done as a transaction that passes through several states
- Transactions are executed in concurrent fashion for better throughput
- Concurrent execution of transactions raise serializability issues that need to be addressed

Module Recap

- A task in a database is done as a transaction that passes through several states
- Transactions are executed in concurrent fashion for better throughput
- Concurrent execution of transactions raise serializability issues that need to be addressed
- All schedules may not satisfy ACID properties

Module Objectives

- To understand the issues that arise when two or more transactions work concurrently

Module Objectives

- To understand the issues that arise when two or more transactions work concurrently
- To introduce the notions of Serializability that ensure schedules for transactions that may run in concurrent fashion but still guarantee and serial behavior

Module Objectives

- To understand the issues that arise when two or more transactions work concurrently
- To introduce the notions of Serializability that ensure schedules for transactions that may run in concurrent fashion but still guarantee and serial behavior
- To analyze the conditions, called conflicts, that need to be honored to attain Serializable schedules

Module Outline

- Serializability

Module Outline

- Serializability
- Conflict Serializability

Serializability

Serializability

- **Assumption**: Each transaction preserves database consistency

Serializability

- **Assumption**: Each transaction preserves database consistency
- Thus, serial execution of a set of transactions preserves database consistency

Serializability

- **Assumption**: Each transaction preserves database consistency
- Thus, serial execution of a set of transactions preserves database consistency
- A (possibly concurrent) schedule is serializable if it is equivalent to a serial schedule

Serializability

- **Assumption**: Each transaction preserves database consistency
- Thus, serial execution of a set of transactions preserves database consistency
- A (possibly concurrent) schedule is serializable if it is equivalent to a serial schedule
- Different forms of schedule equivalence give rise to the notions of:

Serializability

- **Assumption**: Each transaction preserves database consistency
- Thus, serial execution of a set of transactions preserves database consistency
- A (possibly concurrent) schedule is serializable if it is equivalent to a serial schedule
- Different forms of schedule equivalence give rise to the notions of:
  - a) Conflict Serializability

Serializability

- **Assumption**: Each transaction preserves database consistency
- Thus, serial execution of a set of transactions preserves database consistency
- A (possibly concurrent) schedule is serializable if it is equivalent to a serial schedule
- Different forms of schedule equivalence give rise to the notions of:
  - a) Conflict Serializability
  - b) View Serializability

Recap Schedule 3: Serializable

- Let T1 and T2 be the transactions defined previously. The following schedule is not a serial schedule, but it is equivalent to Schedule 1

A	B	A+B	Transaction	Remarks
100	200	300	@ Start	
50	200	250	T1, write A	
45	200	245	T2, write A	
45	250	295	T1, write B	@ Commit
45	255	300	T2, write B	@Commit

Note: In schedules 1, 2 and 3, the sum 'A + B' is preserved

Recap Schedule 4: Not Serializable

- The following concurrent schedule does not preserve the sum of 'A + B'

Simplified View of Transactions

- We ignore operations other than `read` and `write` instructions

Simplified View of Transactions

- We ignore operations other than `read` and `write` instructions
- Other operations happen in memory (are temporary in nature) and (mostly) do not affect the state of the database

Simplified View of Transactions

- We ignore operations other than `read` and `write` instructions
- Other operations happen in memory (are temporary in nature) and (mostly) do not affect the state of the database
- This is a simplifying assumption for analysis

Simplified View of Transactions

- We ignore operations other than `read` and `write` instructions
- Other operations happen in memory (are temporary in nature) and (mostly) do not affect the state of the database
- This is a simplifying assumption for analysis
- We assume that transactions may perform arbitrary computations on data in local buffers in between reads and writes

Simplified View of Transactions

- We ignore operations other than `read` and `write` instructions
- Other operations happen in memory (are temporary in nature) and (mostly) do not affect the state of the database
- This is a simplifying assumption for analysis
- We assume that transactions may perform arbitrary computations on data in local buffers in between reads and writes
- Our simplified schedules consist of only `read` and `write` instructions

Conflicting Instructions

- Let I_i and I_j be two Instructions from transactions T_i and T_j respectively

Conflicting Instructions

- Let I_i and I_j be two Instructions from transactions T_i and T_j respectively
- Instructions I_i and I_j **conflict** if and only if there exists some item Q accessed by both I_i and I_j, and at least one of these instructions write to Q

Conflicting Instructions

- Let I_i and I_j be two Instructions from transactions T_i and T_j respectively
- Instructions I_i and I_j **conflict** if and only if there exists some item Q accessed by both I_i and I_j, and at least one of these instructions write to Q
  - a) $I_i = \text{read}(Q)$, $I_j = \text{read}(Q)$. I_i and I_j **don't conflict**

Conflicting Instructions

- Let I_i and I_j be two Instructions from transactions T_i and T_j respectively
- Instructions I_i and I_j **conflict** if and only if there exists some item Q accessed by both I_i and I_j, and at least one of these instructions write to Q
  - a) $I_i = \text{read}(Q)$, $I_j = \text{read}(Q)$. I_i and I_j **don't conflict**
  - b) $I_i = \text{read}(Q)$, $I_j = \text{write}(Q)$. They **conflict**

Conflicting Instructions

- Let I_i and I_j be two Instructions from transactions T_i and T_j respectively
- Instructions I_i and I_j **conflict** if and only if there exists some item Q accessed by both I_i and I_j, and at least one of these instructions write to Q
  - a) $I_i = \text{read}(Q)$, $I_j = \text{read}(Q)$. I_i and I_j **don't conflict**
  - b) $I_i = \text{read}(Q)$, $I_j = \text{write}(Q)$. They **conflict**
  - c) $I_i = \text{write}(Q)$, $I_j = \text{read}(Q)$. They **conflict**

Conflicting Instructions

- Let I_i and I_j be two Instructions from transactions T_i and T_j respectively
- Instructions I_i and I_j **conflict** if and only if there exists some item Q accessed by both I_i and I_j, and at least one of these instructions write to Q
  - a) $I_i = \text{read}(Q)$, $I_j = \text{read}(Q)$. I_i and I_j **don't conflict**
  - b) $I_i = \text{read}(Q)$, $I_j = \text{write}(Q)$. They **conflict**
  - c) $I_i = \text{write}(Q)$, $I_j = \text{read}(Q)$. They **conflict**
  - d) $I_i = \text{write}(Q)$, $I_j = \text{write}(Q)$. They **conflict**

Conflicting Instructions

- Let I_i and I_j be two Instructions from transactions T_i and T_j respectively
- Instructions I_i and I_j **conflict** if and only if there exists some item Q accessed by both I_i and I_j, and at least one of these instructions write to Q
  - a) $I_i = \text{read}(Q)$, $I_j = \text{read}(Q)$. I_i and I_j **don't conflict**
  - b) $I_i = \text{read}(Q)$, $I_j = \text{write}(Q)$. They **conflict**
  - c) $I_i = \text{write}(Q)$, $I_j = \text{read}(Q)$. They **conflict**
  - d) $I_i = \text{write}(Q)$, $I_j = \text{write}(Q)$. They **conflict**
- Intuitively, a conflict between I_i and I_j forces a (logical) temporal order between them

Conflicting Instructions

- Let I_i and I_j be two Instructions from transactions T_i and T_j respectively
- Instructions I_i and I_j **conflict** if and only if there exists some item Q accessed by both I_i and I_j, and at least one of these instructions write to Q
  - a) $I_i = \text{read}(Q)$, $I_j = \text{read}(Q)$. I_i and I_j **don't conflict**
  - b) $I_i = \text{read}(Q)$, $I_j = \text{write}(Q)$. They **conflict**
  - c) $I_i = \text{write}(Q)$, $I_j = \text{read}(Q)$. They **conflict**
  - d) $I_i = \text{write}(Q)$, $I_j = \text{write}(Q)$. They **conflict**
- Intuitively, a conflict between I_i and I_j forces a (logical) temporal order between them
- If I_i and I_j are consecutive in a schedule and they do not conflict, their results would remain the same even if they had been interchanged in the schedule

Conflict Serializability

Conflict Serializability

- If a schedule S can be transformed into a schedule S' by a series of swaps of non-conflicting instructions, we say that S and S' are **conflict equivalent**

Conflict Serializability

- If a schedule S can be transformed into a schedule S' by a series of swaps of non-conflicting instructions, we say that S and S' are **conflict equivalent**
- We say that a schedule S is **conflict serializable** if it is conflict equivalent to a serial schedule

Conflict Serializability (2)

- Schedule 3 can be transformed into Schedule 6, a serial schedule where T_2 follows T_1, by a series of swaps of non-conflicting instructions:

Conflict Serializability (2)

- Schedule 3 can be transformed into Schedule 6, a serial schedule where T_2 follows T_1, by a series of swaps of non-conflicting instructions:
  - Swap T_1.read(B) and T_2.write(A)

Conflict Serializability (2)

- Schedule 3 can be transformed into Schedule 6, a serial schedule where T_2 follows T_1, by a series of swaps of non-conflicting instructions:
  - Swap T_1.read(B) and T_2.write(A)
  - Swap T_1.read(B) and T_2.read(A)

Conflict Serializability (2)

- Schedule 3 can be transformed into Schedule 6, a serial schedule where T_2 follows T_1, by a series of swaps of non-conflicting instructions:
  - Swap T_1.read(B) and T_2.write(A)
  - Swap T_1.read(B) and T_2.read(A)
  - Swap T_1.write(B) and T_2.write(A)

Conflict Serializability (2)

- Schedule 3 can be transformed into Schedule 6, a serial schedule where T_2 follows T_1, by a series of swaps of non-conflicting instructions:
  - Swap T_1.read(B) and T_2.write(A)
  - Swap T_1.read(B) and T_2.read(A)
  - Swap T_1.write(B) and T_2.write(A)
  - Swap T_1.write(B) and T_2.read(A)

Conflict Serializability (2)

- Schedule 3 can be transformed into Schedule 6, a serial schedule where T_2 follows T_1, by a series of swaps of non-conflicting instructions:
  - Swap T_1.read(B) and T_2.write(A)
  - Swap T_1.read(B) and T_2.read(A)
  - Swap T_1.write(B) and T_2.write(A)
  - Swap T_1.write(B) and T_2.read(A)
- These swaps do not conflict as they work with different items (A or B) in different transactions

Conflict Serializability (3)

- Example of a schedule that is **not** conflict serializable:

T_3	T_4
`read(Q)`	
	`write(Q)`
`write(Q)`	

Conflict Serializability (3)

- Example of a schedule that is **not** conflict serializable:

T_3	T_4
read(Q)	
	write(Q)
write(Q)	

- We are unable to swap instructions in the above schedule to obtain either the serial schedule $\langle T_3, T_4 \rangle$, or the serial schedule $\langle T_4, T_3 \rangle$

Example: Bad Schedule

Example

- Consider two transactions updating account balances:

Example: Bad Schedule

Example

- Consider two transactions updating account balances:
- Schedule S: `r1(A), r2(A), w1(A), w2(A), r2(B), w2(B)`

Example: Bad Schedule

Example

- Consider two transactions updating account balances:
- Schedule S: `r1(A), r2(A), w1(A), w2(A), r2(B), w2(B)`
- Suppose: A starts with $200, and account B starts with $100

Example: Bad Schedule

Example

- Consider two transactions updating account balances:
- Schedule S: `r1(A), r2(A), w1(A), w2(A), r2(B), w2(B)`
- Suppose: A starts with $200, and account B starts with $100

- Schedule S is very bad! We withdrew $100 from account A, but somehow the database has recorded that our account now holds $201!

Example: Bad Schedule (2)

- Ideal schedule is serial:

Example: Bad Schedule (2)

- Ideal schedule is serial:
  - Serial schedule 1: `r1(A), w1(A), r2(A), w2(A), r2(B), w2(B)`

Example: Bad Schedule (2)

- Ideal schedule is serial:
  - Serial schedule 1: `r1(A), w1(A), r2(A), w2(A), r2(B), w2(B)`
  - Serial schedule 2: `r2(A), w2(A), r2(B), w2(B), r1(A), w1(A)`

Example: Bad Schedule (2)

- Ideal schedule is serial:
  - Serial schedule 1: `r1(A), w1(A), r2(A), w2(A), r2(B), w2(B)`
  - Serial schedule 2: `r2(A), w2(A), r2(B), w2(B), r1(A), w1(A)`
- We call a schedule serializable if it has the same effect as some serial schedule regardless of the specific information in the database.

Example: Bad Schedule (2)

- Ideal schedule is serial:
  - Serial schedule 1: `r1(A), w1(A), r2(A), w2(A), r2(B), w2(B)`
  - Serial schedule 2: `r2(A), w2(A), r2(B), w2(B), r1(A), w1(A)`
- We call a schedule serializable if it has the same effect as some serial schedule regardless of the specific information in the database.
- As an example, consider Schedule T, which has swapped the third and fourth operations from S:

Example: Bad Schedule (2)

- Ideal schedule is serial:
  - Serial schedule 1: `r1(A), w1(A), r2(A), w2(A), r2(B), w2(B)`
  - Serial schedule 2: `r2(A), w2(A), r2(B), w2(B), r1(A), w1(A)`
- We call a schedule serializable if it has the same effect as some serial schedule regardless of the specific information in the database.
- As an example, consider Schedule T, which has swapped the third and fourth operations from S:
  - Schedule S: `r1(A), r2(A), w1(A), w2(A), r2(B), w2(B)`

Example: Bad Schedule (2)

- Ideal schedule is serial:
  - Serial schedule 1: `r1(A), w1(A), r2(A), w2(A), r2(B), w2(B)`
  - Serial schedule 2: `r2(A), w2(A), r2(B), w2(B), r1(A), w1(A)`
- We call a schedule serializable if it has the same effect as some serial schedule regardless of the specific information in the database.
- As an example, consider Schedule T, which has swapped the third and fourth operations from S:
  - Schedule S: `r1(A), r2(A), w1(A), w2(A), r2(B), w2(B)`
  - Schedule T: `r1(A), r2(A), w2(A), w1(A), r2(B), w2(B)`

Example: Bad Schedule (2)

- Ideal schedule is serial:
  - Serial schedule 1: `r1(A), w1(A), r2(A), w2(A), r2(B), w2(B)`
  - Serial schedule 2: `r2(A), w2(A), r2(B), w2(B), r1(A), w1(A)`
- We call a schedule serializable if it has the same effect as some serial schedule regardless of the specific information in the database.
- As an example, consider Schedule T, which has swapped the third and fourth operations from S:
  - Schedule S: `r1(A), r2(A), w1(A), w2(A), r2(B), w2(B)`
  - Schedule T: `r1(A), r2(A), w2(A), w1(A), r2(B), w2(B)`
- By first example, the outcome is the same as Serial schedule 1. But that's just a peculiarity of the data.

Example: Bad Schedule (2)

- Ideal schedule is serial:
  - Serial schedule 1: `r1(A), w1(A), r2(A), w2(A), r2(B), w2(B)`
  - Serial schedule 2: `r2(A), w2(A), r2(B), w2(B), r1(A), w1(A)`
- We call a schedule serializable if it has the same effect as some serial schedule regardless of the specific information in the database.
- As an example, consider Schedule T, which has swapped the third and fourth operations from S:
  - Schedule S: `r1(A), r2(A), w1(A), w2(A), r2(B), w2(B)`
  - Schedule T: `r1(A), r2(A), w2(A), w1(A), r2(B), w2(B)`
- By first example, the outcome is the same as Serial schedule 1. But that's just a peculiarity of the data.
- So neither S nor T are serializable.

Example: Good Schedule

Example: Schedule U

- What's a non-serial example of a serializable schedule? We could credit interest to A first, then withdraw the money, then credit interest to B:

Example: Good Schedule

Example: Schedule U

- What's a non-serial example of a serializable schedule? We could credit interest to A first, then withdraw the money, then credit interest to B:
- Schedule U: `r2(A), w2(A), r1(A), w1(A), r2(B), w2(B)`

Example: Good Schedule

Example: Schedule U

- What's a non-serial example of a serializable schedule? We could credit interest to A first, then withdraw the money, then credit interest to B:
- Schedule U: `r2(A), w2(A), r1(A), w1(A), r2(B), w2(B)`
- ▷

  Initial: A = 200, B = 100

Example: Good Schedule

Example: Schedule U

- What's a non-serial example of a serializable schedule? We could credit interest to A first, then withdraw the money, then credit interest to B:
- Schedule U: `r2(A), w2(A), r1(A), w1(A), r2(B), w2(B)`
- ▷

  Initial: A = 200, B = 100
- ▷

  Final: A = 101, B = 100.50

Example: Good Schedule

Example: Schedule U

- What's a non-serial example of a serializable schedule? We could credit interest to A first, then withdraw the money, then credit interest to B:
- Schedule U: `r2(A), w2(A), r1(A), w1(A), r2(B), w2(B)`
- ▷

  Initial: A = 200, B = 100
- ▷

  Final: A = 101, B = 100.50

- Schedule U is conflict serializable to Schedule 2 by swapping non-conflicting pairs:

Example: Good Schedule

Example: Schedule U

- What's a non-serial example of a serializable schedule? We could credit interest to A first, then withdraw the money, then credit interest to B:
- Schedule U: `r2(A), w2(A), r1(A), w1(A), r2(B), w2(B)`
- ▷

  Initial: A = 200, B = 100
- ▷

  Final: A = 101, B = 100.50

- Schedule U is conflict serializable to Schedule 2 by swapping non-conflicting pairs:
  - swap `w1(A)` and `r2(B)`

Example: Good Schedule

Example: Schedule U

- What's a non-serial example of a serializable schedule? We could credit interest to A first, then withdraw the money, then credit interest to B:
- Schedule U: `r2(A), w2(A), r1(A), w1(A), r2(B), w2(B)`
- ▷

  Initial: A = 200, B = 100
- ▷

  Final: A = 101, B = 100.50

- Schedule U is conflict serializable to Schedule 2 by swapping non-conflicting pairs:
  - swap `w1(A)` and `r2(B)`
  - swap `w1(A)` and `w2(B)`

Example: Good Schedule

Example: Schedule U

- What's a non-serial example of a serializable schedule? We could credit interest to A first, then withdraw the money, then credit interest to B:
- Schedule U: `r2(A), w2(A), r1(A), w1(A), r2(B), w2(B)`
- ▷

  Initial: A = 200, B = 100
- ▷

  Final: A = 101, B = 100.50

- Schedule U is conflict serializable to Schedule 2 by swapping non-conflicting pairs:
  - swap `w1(A)` and `r2(B)`
  - swap `w1(A)` and `w2(B)`

Example: Good Schedule

Example: Schedule U

- What's a non-serial example of a serializable schedule? We could credit interest to A first, then withdraw the money, then credit interest to B:
- Schedule U: `r2(A), w2(A), r1(A), w1(A), r2(B), w2(B)`
- ▷

  Initial: A = 200, B = 100
- ▷

  Final: A = 101, B = 100.50

- Schedule U is conflict serializable to Schedule 2 by swapping non-conflicting pairs:
  - swap `w1(A)` and `r2(B)`
  - swap `w1(A)` and `w2(B)`

Serializability Examples

- Are all serializable schedules conflict-serializable? No.

Serializability Examples

- Are all serializable schedules conflict-serializable? No.
- Consider the following schedule for a set of three transactions:

Serializability Examples

- Are all serializable schedules conflict-serializable? No.
- Consider the following schedule for a set of three transactions:
- `w1(A), w2(A), w2(B), w1(B), w3(B)`

Serializability Examples

- Are all serializable schedules conflict-serializable? No.
- Consider the following schedule for a set of three transactions:
- `w1(A), w2(A), w2(B), w1(B), w3(B)`
- We can perform no swaps to this schedule due to writes conflicting on A and B. So this schedule is not conflict-equivalent to any serial schedule.

Serializability Examples

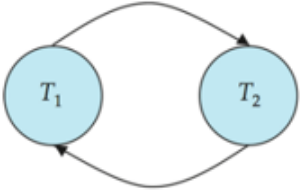
- Are all serializable schedules conflict-serializable? No.
- Consider the following schedule for a set of three transactions:
- `w1(A), w2(A), w2(B), w1(B), w3(B)`
- We can perform no swaps to this schedule due to writes conflicting on A and B. So this schedule is not conflict-equivalent to any serial schedule.
- However, since nobody ever reads the values written by the `w1(A)`, `w2(B)`, and `w1(B)` operations, the schedule has the same outcome as the serial schedule:

Serializability Examples

- Are all serializable schedules conflict-serializable? No.
- Consider the following schedule for a set of three transactions:
- `w1(A), w2(A), w2(B), w1(B), w3(B)`
- We can perform no swaps to this schedule due to writes conflicting on A and B. So this schedule is not conflict-equivalent to any serial schedule.
- However, since nobody ever reads the values written by the `w1(A)`, `w2(B)`, and `w1(B)` operations, the schedule has the same outcome as the serial schedule:
  - `w1(A), w1(B), w2(A), w2(B), w3(B)`

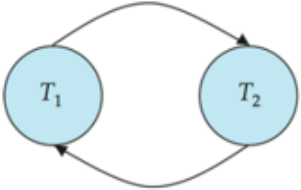
Precedence Graph

- Consider some schedule of a set of transactions $T_1, T_2, \ldots, T_n$



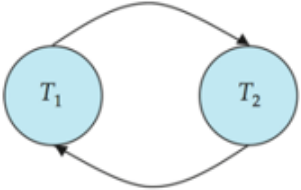
Precedence Graph

- Consider some schedule of a set of transactions $T_1, T_2, \ldots, T_n$
- **Precedence Graph**



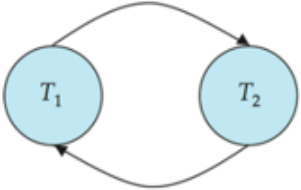
Precedence Graph

- Consider some schedule of a set of transactions $T_1, T_2, \ldots, T_n$
- **Precedence Graph**
  - A directed graph where the vertices are the transactions (names)



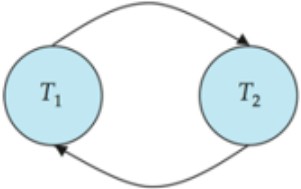
Precedence Graph

- Consider some schedule of a set of transactions $T_1, T_2, \ldots, T_n$
- **Precedence Graph**
  - A directed graph where the vertices are the transactions (names)
  - We draw an arc from T_i to T_j if the two transactions conflict, and T_i accessed the data item on which the conflict arose earlier



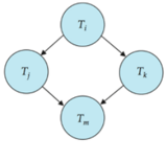
Precedence Graph

- Consider some schedule of a set of transactions $T_1, T_2, \ldots, T_n$
- **Precedence Graph**
  - A directed graph where the vertices are the transactions (names)
  - We draw an arc from T_i to T_j if the two transactions conflict, and T_i accessed the data item on which the conflict arose earlier
  - We may label the arc by the item that was accessed



Testing for Conflict Serializability

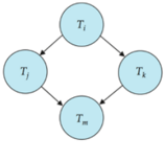
- A schedule is conflict serializable if and only if its precedence graph is **acyclic**



(a)

Testing for Conflict Serializability

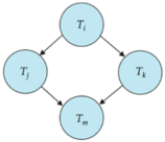
- A schedule is conflict serializable if and only if its precedence graph is **acyclic**
- Cycle-detection algorithms exist which take $\mathcal{O}(n^2)$ time, where n is the number of vertices in the graph



(a)

Testing for Conflict Serializability

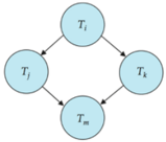
- A schedule is conflict serializable if and only if its precedence graph is **acyclic**
- Cycle-detection algorithms exist which take $\mathcal{O}(n^2)$ time, where n is the number of vertices in the graph
- (Better algorithms take $\mathcal{O}(n + e)$ where e is the number of edges)



(a)

Testing for Conflict Serializability

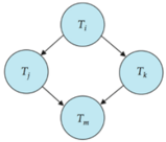
- A schedule is conflict serializable if and only if its precedence graph is **acyclic**
- Cycle-detection algorithms exist which take $\mathcal{O}(n^2)$ time, where n is the number of vertices in the graph
- (Better algorithms take $\mathcal{O}(n + e)$ where e is the number of edges)
- If precedence graph is acyclic, the serializability order can be obtained by a **topological sorting** of the graph



(a)

Testing for Conflict Serializability

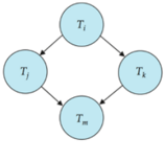
- A schedule is conflict serializable if and only if its precedence graph is **acyclic**
- Cycle-detection algorithms exist which take $\mathcal{O}(n^2)$ time, where n is the number of vertices in the graph
- (Better algorithms take $\mathcal{O}(n + e)$ where e is the number of edges)
- If precedence graph is acyclic, the serializability order can be obtained by a **topological sorting** of the graph
  - That is, a linear order consistent with the partial order of the graph.



(a)

Testing for Conflict Serializability

- A schedule is conflict serializable if and only if its precedence graph is **acyclic**
- Cycle-detection algorithms exist which take $\mathcal{O}(n^2)$ time, where n is the number of vertices in the graph
- (Better algorithms take $\mathcal{O}(n + e)$ where e is the number of edges)
- If precedence graph is acyclic, the serializability order can be obtained by a **topological sorting** of the graph
  - That is, a linear order consistent with the partial order of the graph.
- For example, a serializability order for the schedule (a) would be one of either (b) or (c)



(a)

Testing for Conflict Serializability (2)

- Build a directed graph, with a vertex for each transaction.

Testing for Conflict Serializability (2)

- Build a directed graph, with a vertex for each transaction.
- Go through each operation of the schedule.

Testing for Conflict Serializability (2)

- Build a directed graph, with a vertex for each transaction.
- Go through each operation of the schedule.
- If the operation is $w_i(X)$, find each subsequent operation $r_j(X)$ or $w_j(X)$. Add a directed edge from T_i to T_j.

Testing for Conflict Serializability (2)

- Build a directed graph, with a vertex for each transaction.
- Go through each operation of the schedule.
- If the operation is $w_i(X)$, find each subsequent operation $r_j(X)$ or $w_j(X)$. Add a directed edge from T_i to T_j.
- If the operation is $r_i(X)$, find each subsequent write $w_j(X)$. Add a directed edge from T_i to T_j.

Testing for Conflict Serializability (2)

- Build a directed graph, with a vertex for each transaction.
- Go through each operation of the schedule.
- If the operation is $w_i(X)$, find each subsequent operation $r_j(X)$ or $w_j(X)$. Add a directed edge from T_i to T_j.
- If the operation is $r_i(X)$, find each subsequent write $w_j(X)$. Add a directed edge from T_i to T_j.
- The schedule is conflict-serializable if and only if the resulting directed graph is acyclic.

Testing for Conflict Serializability (2)

- Build a directed graph, with a vertex for each transaction.
- Go through each operation of the schedule.
- If the operation is $w_i(X)$, find each subsequent operation $r_j(X)$ or $w_j(X)$. Add a directed edge from T_i to T_j.
- If the operation is $r_i(X)$, find each subsequent write $w_j(X)$. Add a directed edge from T_i to T_j.
- The schedule is conflict-serializable if and only if the resulting directed graph is acyclic.
- Moreover, we can perform a topological sort on the graph to discover the serial schedule to which the schedule is conflict-equivalent.

Testing for Conflict Serializability (3)

Example

- Consider schedule: `w1(A), r2(A), w1(B), w3(C), r2(C), r4(B), w2(D), w4(E), r5(D), w5(E)`

Testing for Conflict Serializability (3)

Example

- Consider schedule: `w1(A), r2(A), w1(B), w3(C), r2(C), r4(B), w2(D), w4(E), r5(D), w5(E)`
- Edges:

Testing for Conflict Serializability (3)

Example

- Consider schedule: `w1(A), r2(A), w1(B), w3(C), r2(C), r4(B), w2(D), w4(E), r5(D), w5(E)`
- Edges:
  - `w1(A)` $\rightarrow$ `r2(A)` $\Rightarrow T_1 \rightarrow T_2$

Testing for Conflict Serializability (3)

Example

- Consider schedule: `w1(A), r2(A), w1(B), w3(C), r2(C), r4(B), w2(D), w4(E), r5(D), w5(E)`
- Edges:
  - `w1(A)` $\rightarrow$ `r2(A)` $\Rightarrow T_1 \rightarrow T_2$
  - `w1(B)` $\rightarrow$ `r4(B)` $\Rightarrow T_1 \rightarrow T_4$

Testing for Conflict Serializability (3)

Example

- Consider schedule: `w1(A), r2(A), w1(B), w3(C), r2(C), r4(B), w2(D), w4(E), r5(D), w5(E)`
- Edges:
  - `w1(A)` $\rightarrow$ `r2(A)` $\Rightarrow T_1 \rightarrow T_2$
  - `w1(B)` $\rightarrow$ `r4(B)` $\Rightarrow T_1 \rightarrow T_4$
  - `w3(C)` $\rightarrow$ `r2(C)` $\Rightarrow T_3 \rightarrow T_2$

Testing for Conflict Serializability (3)

Example

- Consider schedule: `w1(A), r2(A), w1(B), w3(C), r2(C), r4(B), w2(D), w4(E), r5(D), w5(E)`
- Edges:
  - `w1(A)` $\rightarrow$ `r2(A)` $\Rightarrow T_1 \rightarrow T_2$
  - `w1(B)` $\rightarrow$ `r4(B)` $\Rightarrow T_1 \rightarrow T_4$
  - `w3(C)` $\rightarrow$ `r2(C)` $\Rightarrow T_3 \rightarrow T_2$
  - `w2(D)` $\rightarrow$ `r5(D)` $\Rightarrow T_2 \rightarrow T_5$

Testing for Conflict Serializability (3)

Example

- Consider schedule: `w1(A), r2(A), w1(B), w3(C), r2(C), r4(B), w2(D), w4(E), r5(D), w5(E)`
- Edges:
  - `w1(A)` $\rightarrow$ `r2(A)` $\Rightarrow T_1 \rightarrow T_2$
  - `w1(B)` $\rightarrow$ `r4(B)` $\Rightarrow T_1 \rightarrow T_4$
  - `w3(C)` $\rightarrow$ `r2(C)` $\Rightarrow T_3 \rightarrow T_2$
  - `w2(D)` $\rightarrow$ `r5(D)` $\Rightarrow T_2 \rightarrow T_5$
  - `w4(E)` $\rightarrow$ `w5(E)` $\Rightarrow T_4 \rightarrow T_5$

Testing for Conflict Serializability (3)

Example

- Consider schedule: `w1(A), r2(A), w1(B), w3(C), r2(C), r4(B), w2(D), w4(E), r5(D), w5(E)`
- Edges:
  - `w1(A)` $\rightarrow$ `r2(A)` $\Rightarrow T_1 \rightarrow T_2$
  - `w1(B)` $\rightarrow$ `r4(B)` $\Rightarrow T_1 \rightarrow T_4$
  - `w3(C)` $\rightarrow$ `r2(C)` $\Rightarrow T_3 \rightarrow T_2$
  - `w2(D)` $\rightarrow$ `r5(D)` $\Rightarrow T_2 \rightarrow T_5$
  - `w4(E)` $\rightarrow$ `w5(E)` $\Rightarrow T_4 \rightarrow T_5$

- This graph has no cycles. One way to topologically sort is $T_3 - T_1 - T_4 - T_2 - T_5$. Serial schedule equivalent: `w3(C), w1(A), w1(B), r4(B), w4(E), r2(A), r2(C), w2(D), r5(D), w5(E)`

Module Summary

- Understood the issues that arise when two or more transactions work concurrently

Slides used in this presentation are borrowed from `http://db-book.com/` *with kind permission of the authors.*
Edited and new slides are marked with 'PPD'.

Module Summary

- Understood the issues that arise when two or more transactions work concurrently
- Learnt the forms of serializability in terms of conflict and view serializability

Slides used in this presentation are borrowed from `http://db-book.com/` *with kind permission of the authors.*
Edited and new slides are marked with 'PPD'.

Module Summary

- Understood the issues that arise when two or more transactions work concurrently
- Learnt the forms of serializability in terms of conflict and view serializability
- Acyclic precedence graph can ensure conflict serializability

Slides used in this presentation are borrowed from `http://db-book.com/` *with kind permission of the authors.*
Edited and new slides are marked with 'PPD'.